

Lab 2: Five-Stage Pipelined Processor

1 Introduction

The purpose of this lab was to gain hands-on experience with topics covered in lecture, including instruction set architectures (ISAs), basic pipelined processor microarchitecture, hazard handling, and processor-memory interfacing, by designing and evaluating two pipelined processor microarchitectures for the TinyRV2 ISA. This lab emphasizes key computer architecture concepts from lecture such as stalling, data and control hazards, cycle time, and analyzing processor performance through cycles per instruction (CPI) and average latency. Our implementation followed an incremental development approach, beginning with a baseline processor and then creating an alternative design with additional optimizations, building off the control/-datapath split shown in figure 4 as learned in Lab 1. The baseline design handled data hazards through stalling, pausing the pipeline when an instruction depended on a value that had not yet been computed. The alternative design incorporated bypassing (forwarding), allowing dependent values to be forwarded from later stages to earlier stages of the pipeline, reducing stalls while maintaining correctness. Both designs also included squashing logic to manage control hazards. Throughout the lab, we worked with the TinyRV2 ISA both for design and testing, using instruction fields to generate control signals in the control unit and writing TinyRV2 assembly programs for verification. Performance evaluation showed that the alternative design reduced the number of cycles required for instruction sequences with dependencies, resulting in roughly a 20% decrease in CPI for tasks such as a convolutional filter benchmark (7660 cycles versus 9540 cycles on the baseline), while programs with fewer hazards showed minimal difference. These results demonstrate that bypassing provides significant performance benefits with minimal additional hardware and energy cost.

2 Alternative Design

Our baseline processor handled data hazards using hardware stalling. This ensured correctness but reduced throughput, since dependent instructions often waited several cycles before resuming. Stalls were triggered in cases of read-after-write (RAW), hazards, as shown in figure 5, where a later instruction required a value that had not yet been written back to the register file. In these situations, pipeline stages were forced to hold instructions for extra cycles, leading to wasted execution bandwidth.

The alternative design improved on this by adding bypassing (forwarding) paths. Bypassing allowed results from the execute (E), memory (M), and writeback (W) stages of the pipeline to be forwarded directly to the decode (D) stage, eliminating the need to wait for register file updates in the writeback (W) stage. This greatly reduced wasted cycles and improved overall throughput. The only case where stalling remained necessary was with load-use hazards: since load data is only available at the end of the memory stage, an instruction immediately following a load that depends on its result must stall for one cycle.

As shown in Figure 1, we implemented three bypass paths: from the end of the execute stage to the decode stage, from the end of the memory stage to the decode stage, and from the beginning of the writeback stage to the decode stage. These values were routed through two four-input bypass multiplexers placed before the operand selection logic in the decode stage. Each mux selected among the register file output and the three bypass sources, forwarding the correct operand when a dependency was detected. This required a few hardware additions: two new muxes, a few comparators, and additional control signals.

The control logic was updated accordingly. Only RAW and control hazards were relevant; structural and WAW/WAR hazards do not occur in this pipeline. Control hazards continued to be handled by squashing, while the stalling logic for RAW hazards was revised. The processor now stalls only for load-use dependencies. For all other RAW hazards, the control logic detects if the source registers in decode match the destination registers in later stages. If so, the appropriate bypass path is enabled, and the instruction proceeds without stalling. This change significantly reduced the number of cycles lost to hazards compared to the baseline design.

Bypassing was chosen for the alternative processor because it offers a large performance improvement with minimal hardware cost. Unlike stalling, which delays dependent instructions until register writeback, bypassing forwards values directly from the execute, memory, or writeback stages. This eliminates unnecessary pipeline bubbles. For example, in a small assembly sequence with arithmetic dependencies, the baseline

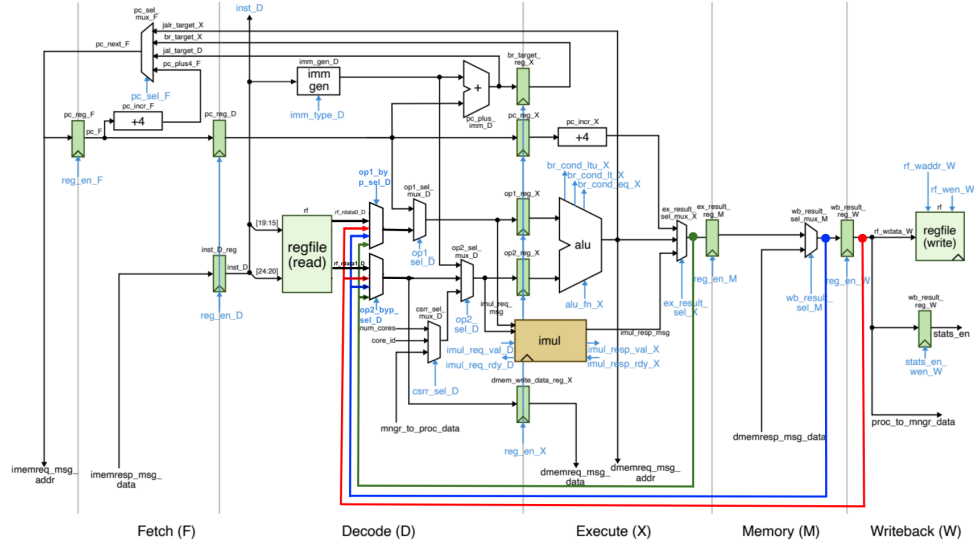


Figure 1: Alternative datapath showing added forwarding and pipeline control logic.

stalling design required nine cycles to complete, while the bypassing design required only three cycles (excluding pipeline warm-up). This threefold reduction clearly demonstrates the effectiveness of bypassing in improving throughput with only a modest increase in hardware complexity.

3 Testing Strategy

We used the provided `pytest` framework along with directed and integration tests. At the unit level, we tested critical modules such as the ALU, register file, and control logic using directed inputs and expected outputs. For example, ALU tests covered addition, subtraction, logic operations, shifts, and comparisons, including edge cases such as overflow and signed vs. unsigned behavior. We also tested the immediate generation unit by including test cases to cover all the immediate types. We used incremental and test driven approaches for completing both the baseline and the alternative design. We verified our implementation of each instruction before continuing to the next one by running the provided basic test and a number of several simple directed value tests we wrote, as shown in table 2, before starting our development. To ensure the tests we wrote were correct, we tested the testcases against the Functional Level (FL) model of the Pipelined processor. The functional level does not require a microarchitecture to compute the results so we used that as a benchmark for our test cases since the FL operation is always guaranteed to be correct. We tested each instruction incrementally and added more test cases to deal with data hazard cases. We also added random value testing which included random delay testing to ensure that the processor could handle all cases and the uncertainty of real-world computation. We combined these directed and random tests into unit test suites for each instruction to verify the correctness of the processor in sequences of instructions. While testing, some test cases did not pass so we used trace outputs and Surfer waveforms to look at what our processor values were at specific clock cycles.

Directed testing was essential to ensuring that our instructions would respond appropriately to read-after-write hazards. Specifically, we added Python assembly-generation test functions(given to us in the release code) for destination-dependent and source-dependent hazards as well as when the source and destination are the same. To make sure that the microarchitecture would either stall or bypass to handle these cases, we varied the number of nops between consecutive instructions from 5 to 0 to test whether, for example, a sub instruction in the baseline design would stall until both source registers have been written by the preceding `csrr` instruction. We also wrote directed value tests to check that the implementation correctly handled edge cases, especially those instructions in which signed/unsigned representations matter. In our test suite, we created directed cases for each instruction type to cover a wide range of corner conditions. For register-register and register-immediate operations, we exercised both operands with values such as zero,

small positives, large positives, small negatives, and large negatives. For memory-related instructions, we applied the same edge cases both to the data being read/written and to the target memory addresses. For jumps and branches, we validated correctness by monitoring program counter (PC) updates. Specifically, we used a register that accumulates markers via `addi` instructions placed at different labeled positions in the program. If control flow behaves as expected, certain markers are skipped, which confirms that the PC was redirected properly. These tests also ensured that the design correctly handles both forward and backward branches. Additionally, we checked that squash signals behaved properly by confirming that only the intended instructions execute following a branch or jump. In addition to directed cases, we incorporated randomized testing to validate correctness across a broader set of scenarios and to better reflect the unpredictable workloads a processor may encounter in practice.

Randomized tests are particularly helpful for uncovering edge conditions or subtle bugs that might slip past directed checks. To generate these tests, we used Python’s random number generation to select operand values and compared the processor’s results against expected outputs computed from those inputs. For control-flow instructions, random testing included inserting a varying number of nops between branches and jumps to ensure the branch logic remained robust. We also experimented with randomized delays, which mimic real hardware latencies such as those from memory stalls or cache misses, and confirmed that the processor appropriately stalls until data becomes available. For the jump instructions specifically, Generative AI was important in writing the random test cases. Finally, we structured both our directed and random tests into modular unit tests, grouped by instruction classes (e.g., register-register, register-immediate, memory). This hierarchy allowed us to verify that no essential datapath components were missing and to confirm the correctness of complex instructions like branches that rely on simpler building blocks such as `addi`. At the highest level, we combined multiple instruction classes in integrated tests, adjusting or removing nops to verify that hazards were resolved correctly and that the processor maintained stable operation under more realistic workloads.

At the integration level, we used assembly test programs like `vvadd-unopt`, `vvadd-opt`, `cmult`, `mfilt`, and `bsearch`. These programs exercised both common instruction mixes and corner cases such as load-use hazards and branching. We relied on the functional-level (FL) processor as a golden reference model to verify correctness. Each program was run on both baseline and alternative implementations, and we confirmed that the outputs matched the reference.

Our overall strategy combined unit testing (to quickly isolate bugs) with integration testing (to validate pipeline behavior). The combination ensured that individual components were correct and that the pipeline as a whole behaved as intended under realistic workloads.

4 Evaluation

We evaluated both the baseline and alternative pipelined processor designs using the provided benchmarks: `vvadd-unopt`, `vvadd-opt`, `cmult`, `bsearch`, and `mfilt`. Each benchmark exercises different instruction patterns and hazard behaviors, allowing us to assess performance under various workloads. The simulator reports instruction count, total cycle count, and CPI for each implementation. Verification mode was used to ensure functional correctness across all benchmarks.

Performance Results. As shown in Table 1, the alternative design consistently achieves lower CPI across most benchmarks. The `vvadd-unopt` benchmark shows a clear improvement, with CPI decreasing from 2.22 in the baseline to 1.34 in the alternative design, which is a reduction of nearly 40%. Similarly, `bsearch` improves from 2.97 to 1.38 (53.5% decrease), and `mfilt` drops from 4.27 to 1.87 (56.2% decrease). These workloads contain frequent read-after-write (RAW) data dependencies, which in the baseline design cause multiple stall cycles since operands must wait to be written back to the register file. The alternative design reduces these stalls by using bypassing logic to forward data directly between pipeline stages, allowing dependent instructions to execute without delay.

For `vvadd-opt`, both designs achieve an identical CPI of 1.10 (0% improvement). This result makes sense because the optimized version of the benchmark unrolls loops to remove dependencies between successive iterations, effectively eliminating data hazards. The `cmult` benchmark shows a smaller improvement, with CPI decreasing slightly from 3.06 to 2.88 (5.9% decrease). This smaller difference occurs because the multiplication unit already manages operand readiness internally and does not require explicit stalling or

bypassing.

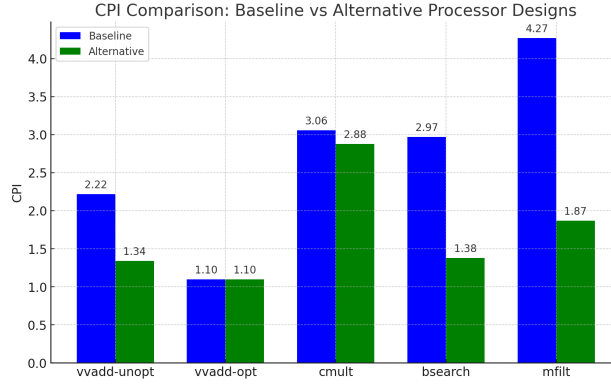


Figure 2: Bar plot of CPI for the evaluated benchmarks.

Cycle Time Analysis. While the alternative design reduces CPI, it incurs a slightly longer cycle time. This increase results from the longer critical path created by the bypassing logic. For example, when an instruction in the execute (X) stage forwards its result to the decode (D) stage, the signal must propagate through additional multiplexers and control logic, extending the combinational delay. In contrast, the baseline design isolates each stage more cleanly, resulting in a shorter critical path. Despite this, the CPI improvements outweigh the slower clock frequency, leading to faster total execution for most workloads.

Area Analysis. The alternative design introduces modest area overhead. The added bypass multiplexers, control signals, and wiring connections marginally increase the datapath size. However, since the pipeline depth, register file, and functional units remain unchanged, the overall area growth is minor. This is a well-justified tradeoff: a small increase in hardware complexity provides a substantial performance gain by minimizing stalls.

Energy Analysis. In terms of energy efficiency, the alternative design performs better overall. The baseline processor wastes dynamic power during stalls, as pipeline registers and control signals toggle without performing useful computation. By contrast, the alternative design allows more instructions to execute per cycle, improving energy per instruction. Although the added bypass logic contributes slightly to static and switching power, the reduction in wasted cycles compensates for it, yielding a lower overall energy cost per instruction.

5 Conclusion

In this lab, we implemented and evaluated a five-stage pipelined RISC-V processor, comparing a baseline stalling design with an alternative version incorporating data forwarding. The alternative achieved lower CPI and higher throughput on most benchmarks, improving performance by up to 56% on `mfilt` and `bsearch`, while `vvadd-opt` showed no change due to its lack of data dependencies. On average, CPI improved by about 31%, reducing total cycles by roughly 37%. These results demonstrate that forwarding effectively mitigates data hazards and improves pipeline efficiency, though at the cost of slightly increased area and cycle time. The performance benefits are most pronounced for workloads with frequent dependent operations, while memory-bound or dependency-free programs gain little. Overall, this lab highlighted the importance of balancing complexity and performance in pipeline design and showed how concepts like hazards, stalling, and forwarding translate into measurable architectural improvements. Importantly, it also provides insight into how design choices affect different workloads, emphasizing that optimizations like forwarding are most valuable when dependencies are frequent. Looking forward, these enhancements will help reduce pipeline stalls in Lab 4 when integrating this processor with cache memory in a multicore system, particularly improving the performance of loop-heavy parallel workloads that would otherwise suffer from frequent branch mispredictions, and setting the stage for exploring additional optimizations such as branch prediction and cache tuning.

6 Work Distribution

Both team members made significant contributions to the design, implementation, and debugging of the pipeline processor. While responsibilities were divided to focus on different subsystems, the project was ultimately carried out collaboratively, with design, integration testing and final evaluations performed together. Generative AI tools were used to assist in debugging Verilog syntax issues, such as missing signal declarations, incorrect module instantiations, mismatched signal widths and undefined wires. AI tools were also used to enhance the clarity and flow of the lab report by correcting grammatical errors, improving phrasing, and suggesting ways to better articulate ideas. We also used AI for brainstorming and automating some test case generation, but all architectural design decisions, datapath/control logic implementation, and debugging were performed by the team. Jamayne was primarily responsible for implementing the register-register and register-immediate instructions, as well as developing the datapath and control path for these instruction classes. Jamayne also created the directed and randomized test cases for validating instruction functionality, leveraging both manual design and AI-generated assistance for coverage expansion. In addition, Jamayne documented the testing methodology, drafting the Conclusion and Work Distribution sections, and producing the figures and diagrams used in the report. Md focused on the alternative design exploration and took charge of developing the datapath and control logic for branching and jumping instructions, ensuring that control flow and hazard handling behaved correctly. On the documentation side, Md authored the Introduction, the Design Explanation, and the Evaluation sections of the report, where the key architectural trade-offs and performance implications were discussed.

7 Appendix

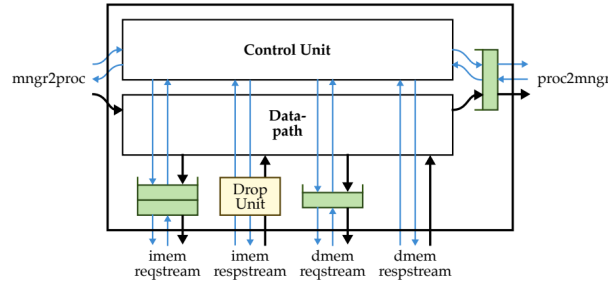


Figure 3: Processor Datapath and Control Composition.

Benchmark	Base			Alt			% Improvement (CPI)
	Cycles	Insts	CPI	Cycles	Insts	CPI	
vvadd-unopt	2014	909	2.22	1214	909	1.34	39.6%
vvadd-opt	589	534	1.10	589	534	1.10	0.0%
cmult	5226	1709	3.06	4930	1709	2.88	5.9%
bsearch	4537	1529	2.97	2116	1529	1.38	53.5%
mfilt	5779	1352	4.27	2525	1352	1.87	56.2%

Table 1: Performance comparison of baseline and alternative implementations with CPI Improvements.

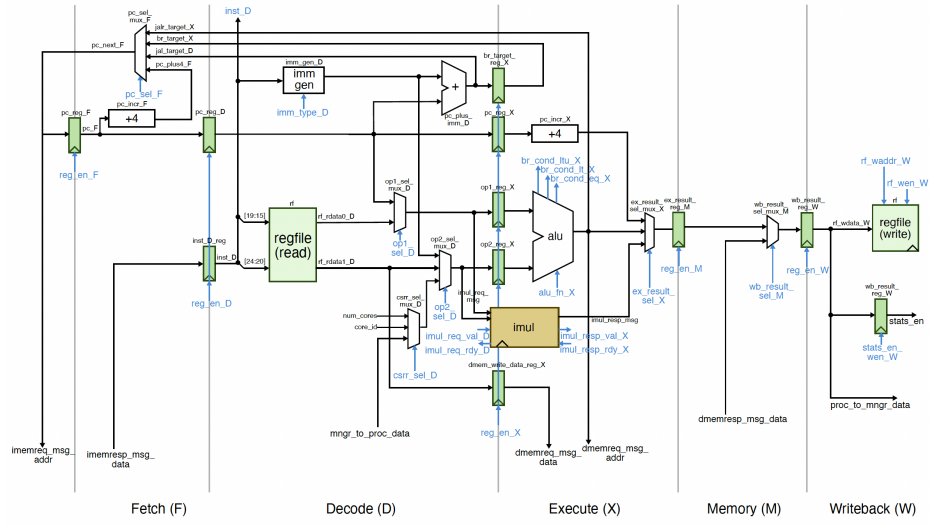


Figure 4: Baseline datapath of the processor design.

Table 2: Total number of tests per instruction Category

Instruction Category	Number of Tests
RR (Register-Register)	80
RIMM (Register-Immediate)	36
Memory	10
Jump	14
CSR	5
Branch	44

```

slli x24, x11, 2
add  x16, x4, x24
lw   x17, 0(x16)
bne  x9, x17, two
add  x16, x5, x24
lw   x18, 0(x16)
add  x15, x2, x25
sw   x18, 0(x15)
addi x13, x0, 1

```

The code snippet illustrates RAW hazards with red arrows and circles. Red circles highlight the values of registers x24, x16, x17, x18, and x15. Red arrows show the flow of data from these registers to subsequent instructions: x24 is used in the second and fourth instructions; x16 is used in the third and fifth instructions; x17 is used in the fourth instruction; x18 is used in the sixth and seventh instructions; and x15 is used in the seventh instruction.

Figure 5: Code Snippet With RAW Hazards.